



Learning dynamics, pattern recognition capability and interpretability of the Tsetlin Machine

Olga Tarasyuk^{a,*}, Anatoliy Gorbenko^b, Tousif Rahman^a, Lei Jiao^c,
Ole-Christoffer Granmo^c, Rishad Shafik^a, Alex Yakovlev^a

^a School of Engineering, Newcastle University, Newcastle upon Tyne, United Kingdom

^b School of Built Environment, Engineering and Computing, Leeds Beckett University, Leeds, United Kingdom

^c Center for Artificial Intelligence Research, University of Agder, Agder, Norway

ARTICLE INFO

Keywords:

Explainable logic-based machine-learning
Learning automata
Tsetlin Machine
Pattern recognition
Learning dynamics
Visualization
Interpretability
Hyperparameters optimisation
Generalisation and specialisation trade-off
Training completion
Convergence

ABSTRACT

The inability to trace an AI's reasoning process and understand why it makes each decision is known as the black box problem. This remains one of the major barriers to the trusted and widespread use of machine learning in many application domains. The paper explores pattern recognition performance and learning dynamics of the Tsetlin Machine – a new explainable logic-based machine-learning approach. Tsetlin Machine uses a collection of finite-state automata with a unique logic-based learning mechanism and provides a promising alternative to Artificial Neural Networks with several advantages, such as interpretability, low complexity, suitability for hardware implementation and high performance. This work investigates Tsetlin Machine's mechanism for constructing conjunctive clauses from data and their interpretation for pattern recognition on several datasets. We demonstrate that during training the logical clauses learn persistent sub-patterns within the class. Each clause creates a class template by clustering a certain number of similar class samples, combining them through literal-wise logical conjunction (i.e., AND-ing). The number of class samples that each clause combines depends on Tsetlin Machine's hyperparameters. The more class samples that are combined, the more general the clauses become. The paper aims at uncovering how Tsetlin Machine's hyperparameters influence the balance between clause generalization and specialization and how this affects the accuracy of pattern recognition. It also studies the evolution of the machine's internal state, its convergence and training completion.

1. Introduction

Machine learning (ML) has become an essential tool for analyzing complex data and making predictions in various application domains, e. g., health care [1], financial sector [2], cybersecurity [3]. Deep neural networks (DNNs) together with other conventional models such as Random Forests, SVMs and Decision trees have become the prevailing techniques in recent years because of their strong predictive performance. However, these models often suffer from high computational complexity [4], a large number of hyperparameters that need to be finetuned to achieve optimal performance [5] and limited interpretability [6]. In response, there is growing interest in alternative, often more interpretable and compute-efficient ML approaches, e.g. based on automata [7] or logic inference [8]. This paper investigates one such alternative, the Tsetlin Machine (TM), proposed by Granmo in 2018 [9]. The TM builds on Tsetlin Automata (TA) that originates from the

research on collective behaviour of learning automata by Tsetlin [10] and Varshavsky [11] and further developed in [12].

Recent studies have shown that the TM provides a promising alternative to DNNs with several advantages: TM is an interpretable, low-complexity algorithm and has a unique logic-based learning mechanism supporting parallelism and efficient hardware implementation with considerably fewer number of hyperparameters to tune. This makes TM attractive for both embedded applications and hardware acceleration [13,14]. TM has been actively developed over the last few years and has demonstrated competitive accuracy on diverse benchmarks. It has been successfully used for pattern recognition in various applications domains, such as image and signal processing [15,16], speech recognition [17], text categorisation [18], finding immune fingerprints in biomarkers [19], etc. While these studies primarily focused on demonstrating the competitiveness of the TM relative to other ML methods across various application domains, the present paper instead

* Corresponding author.

E-mail address: olga.tarasyuk@newcastle.ac.uk (O. Tarasyuk).

<https://doi.org/10.1016/j.patcog.2025.113028>

Received 17 June 2025; Received in revised form 20 December 2025; Accepted 29 December 2025

Available online 30 December 2025

0031-3203/© 2025 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

investigates aspects that remain under-explored: clause interpretability and internal learning dynamics, which are crucial for understanding TM's performance and pattern recognition capability. Building on earlier systematic efforts to identify optimal hyperparameters [20], we show how the learning sensitivity and voting threshold jointly govern the balance between clause generalisation and specialisation critical for TM performance.

Our approach employs visualisation to interpret the evolution of clause-level patterns throughout the training process towards a converged state as a function of the hyperparameters, thereby extending prior work on the dynamics of TM's learning automata [21]. Furthermore, in contrast to the common practice of inferring training completion from accuracy metrics, which can incur substantial time and energy overheads and may not reliably indicate convergence, we propose a completion criterion based on the TM reaching a clause-level Nash equilibrium, thereby providing further empirical support for recent theoretical work on TM convergence [22].

Lack of explainability and interpretability remains a major barrier to the wider adoption of AI in safety- and mission-critical applications [6]. Explainable AI (XAI) is an active and rapidly developing area of research in which visualization is widely used as a powerful tool for understanding the behaviour of intelligent systems [23]. While existing visualization [24] and explainability frameworks like SHAP or LIME [25] can provide useful explanations to convey how the ML algorithm arrived at a particular prediction (e.g., showing how convolutional neural networks transform input images from layer to layer into a final classification or revealing the importance of different features and their contribution to the final decision through SHAP plots), they struggle to explain the underlying logical structure of the decision-making process in DNNs and other conventional ML models and can still be challenging for non-expert users to interpret. Besides, many XAI frameworks experience issues with scalability, both in terms of computational cost and memory usage, particularly when applied to large datasets or complex ML models. In contrast, the TM offers a form of built-in interpretability: its decision-making is inherently transparent because it is based on explicit, human-readable logical rules represented by clauses. Each clause specifies a conjunction of features necessary for a particular classification decision, effectively capturing sub-patterns of each class. These clause-level rules can be directly visualised and understood without additional computational overheads required by post-hoc methods, thereby offering a more scalable and intuitively interpretable approach.

As the primary benchmark this paper uses the MNIST database of handwritten digits – one the most popular and widely recognized AI benchmarks on pattern recognition, the visual nature of which helps to understand the internal state of TM and provide insights into TM's learning dynamics. The key findings reported in the paper were also verified on a diverse range of other visual and non-visual datasets, including GTSRB, IMDB, Fashion- and Kuzushiji-MNIST.

Although a comprehensive evaluation of TM performance across various datasets is beyond this paper's scope, it is worth mentioning that TM has achieved a peak testing accuracy of 99.4 %, 96.31 % and 91.5 % on MNIST, K-MNIST and F-MNIST datasets respectively [26] and demonstrated competitive results against state-of-the-art ML methods on numerous other benchmarks. However, TM continues to face challenges when applied to highly complex visual data such as CIFAR-100 and ImageNet, where class patterns are less stable and deep learning models currently outperform TM in terms of raw accuracy. This performance gap is partly explained by the fact that such datasets violate a key assumption of stationary stochastic environments, under which learning automata and, by extension, the TM are theoretically proven to converge to optimal behaviour [27,28]. Indeed, in CIFAR-like datasets, the relationship between pixels and semantic labels is less stationary due to greater intra-class variability, and effective classification requires hierarchical abstraction, which is not natively supported by the standard TM architecture. Investigating approaches such as semantic

preprocessing or hybrid architectures to enhance TM performance on complex visual datasets remains an important direction for future research. There has also been recent development in various specialized TM architectures such as *Coalesced TMs* [29] for memory efficient architectures, *Regression TMs* [30] for timeseries regression applications and *Convolutional* and *Composite TMs* [26,31] specializing on image classification. These implementations, available on GitHub [32], have retained the low computational effort and energy efficiency of the original TM while indicating opportunity for scalability towards more complex problems suited to DNNs and CNNs.

Through this work we propose the following research contributions (RCs):

- RC1: Visualization and Interpretation of TM's Internal State. The paper contributes to understanding how the TM learns patterns by visualizing and interpreting the internal state of its learning engine. This is achieved through the analysis of logical clauses constructed by the TM, providing insights into the decision-making process and enabling a better understanding of how patterns are recognized and learned.
- RC2: Impact of Hyperparameters on TM Performance. The paper investigates the effects of hyperparameters on the pattern recognition capabilities of the TM. It demonstrates how adjusting hyperparameter values can balance the generalization and specialization of clauses, thus optimizing training and testing accuracy. This contributes to a deeper understanding of TM's behaviour under different settings, guiding effective hyperparameter tuning.
- RC3: Criteria for Determining Training Completion. The paper outlines criteria for deciding when the training process of the TM is complete, ensuring that the TM has reached a state of stability at clause level, thereby reducing energy consumption through the elimination of unnecessary training iterations and making the approach more suitable for deployment on resource-constrained edge hardware.

The paper is organised as follows. Section 2 overviews the TM architecture and defines its hyperparameters. Section 3 investigates clause interpretability and examines visual templates created by positive and negative clauses. Section 4 discusses TM's ability to find class sub-patterns by clustering and combining similar data samples. Section 5 studies the influence of hyperparameters on clauses generalization ability and TM's classification accuracy. Section 6 explores the evolution of TM internal state, its convergence to the Nash equilibrium and training completion. The last two sections summarise key findings and conclude the paper.

2. TM architecture and hyperparameters

In this section we introduce the architecture of the TM, briefly discuss the concept of learning automata, which is in the very nature of TM, and overview TM's learning algorithm and its hyperparameters. More comprehensive description of this recently new ML algorithm, its detailed mathematical foundation and key differences with other ML approaches can be found in earlier works, e.g., [9,33].

2.1. Tsetlin automaton

The theory on which the TM is based originates from the research on collective behaviour of learning automata by Tsetlin [10]. In the early 1960s, Tsetlin studied how rats found their way in mazes (see Fig. 1) to determine how biological systems learn. He demonstrated that finite automata provided a sufficiently accurate and yet simple mathematical model for this behaviour. Such learning automata with linear tactics were later allocated to a special class called Tsetlin automata (TA) [34].

In the T-maze, a rat faces a binary decision at the junction: turn left or right and learns to associate specific paths with received rewards or

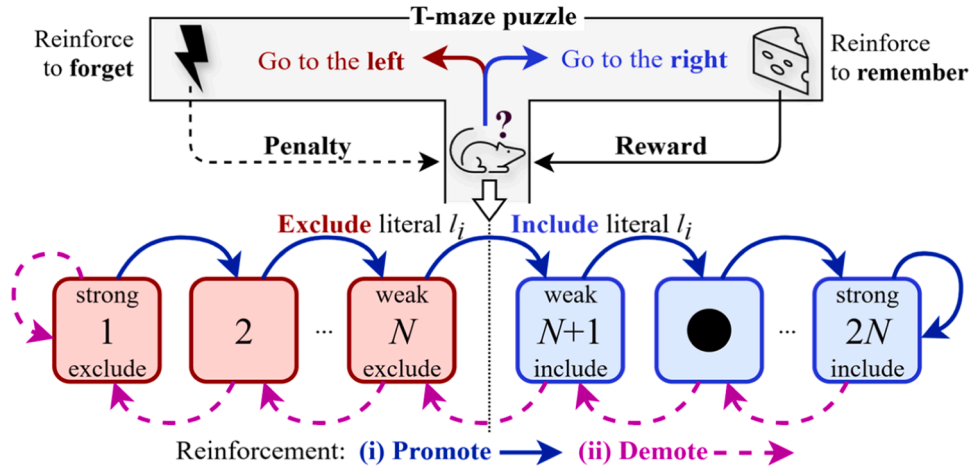


Fig. 1. Tsetlin Automaton (TA) – a mathematical representation of a rat brain: states and transitions.

penalties. Through repeated trials, a rat develops a preference for the rewarding path. TA similarly operates in binary decision environments, making choices between two actions (*include* or *exclude* the certain input feature to/from the conjunctive clause – an inference rule) based on its learned state. This binary decision-making setup in both systems is fundamental to their learning processes and helps simplify the reinforcement mechanism. TA, like a rat, learns by reinforcing actions that lead to favorable outcomes (i.e. the correct classification outcome produced by a logical clause) and discouraging those that do not. Each time the automaton encounters a reward, it strengthens its preference for the corresponding action, much like the rat learns to choose the rewarding path in the maze. The TA behaves like a finite state machine where $2N$ states $\Phi = \{\varphi_1, \varphi_2, \dots, \varphi_{2N}\}$ are divided equally into two actions to “forget” or “remember” to a given decision:

$$a(\varphi) = \begin{cases} 0, & \text{i.e., forget, } \varphi \leq N, \\ 1, & \text{i.e., remember, } N+1 \leq \varphi \leq 2N. \end{cases}$$

Reward and penalty signals $\beta = \{\beta_{\text{Promote}}, \beta_{\text{Demote}}\}$ transition the TA between states Φ (Fig. 1), which allows it to memorize the most common features of the class template.

2.2. Tsetlin machine architecture

Granmo incorporated propositional logic and a game-theoretic bandit driven approach to orchestrate the collective behaviour of TAs and to formulate logical propositions – clauses, which are used to solve classification tasks [9].

Hundreds of such “rat brains” (i.e., TAs), each making one decision, combined through NOT AND operations that relate the Booleanised inputs to their respective TA actions to form logic clauses are referred to as the TM. Fig. 2 shows the general TM architecture. To draw parallels with the common DNN architecture, the TM building blocks were grouped into input, decision (hidden) and output layers.

Input Layer. The learning process for the TM centres on building logical expressions, i.e., TM clauses. These expressions relate the Boolean input variables, also known as Boolean features $X = (x_1, \dots, x_k) \in X$, $X = \{0, 1\}^k$ to the learning elements – the TAs. Some of these variables may be Boolean by nature, e.g., yes-no answers, if-conditions: {true, false}, etc. Others need to be converted to Boolean features using simple ranking or thresholding methods (see Fig. 3) or more advanced significance-driven Booleanisation algorithms [35]. These Boolean features together with their negated counterparts form a set of Boolean literals $L = \{X \cup \neg X\}$ that are then fed to the “decision layer” of the TM.

Decision Layer. This layer in which knowledge is represented as a collection of conjunctive clauses associated with each class label

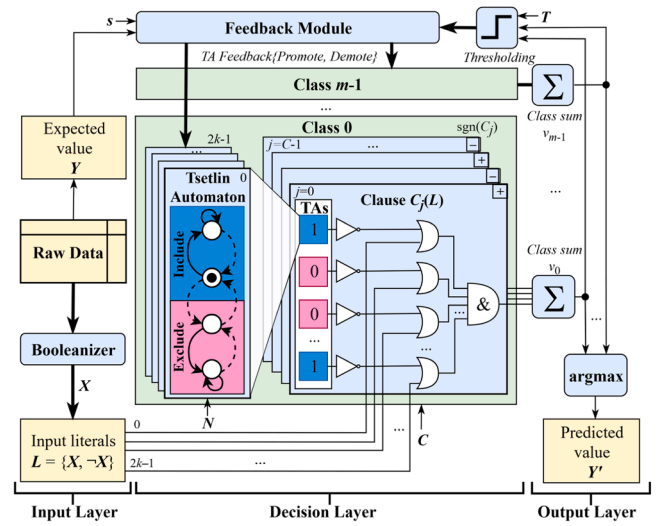


Fig. 2. TM Architecture. TM hyperparameters are: C – number of clauses per class; $2N$ – number of TA states; s – learning sensitivity; T – voting threshold.

highlights the fundamental difference between DNNs and TMs. Basic TM architecture only has a single decision layer between the inputs and outputs. This layer performs the clause computing.

For each literal $\ell \in L$, a dedicated TA decides whether the literal is included in or excluded from a given clause. The action of a TA linked to literal ℓ in clause j for class y is denoted as $a_{y,j,\ell} \in \{0, 1\}$, where $a_{y,j,\ell} = 1$ indicates inclusion and $a_{y,j,\ell} = 0$ indicates exclusion. This masking behaviour can be expressed as:

$$\tilde{\ell}_{y,j}(\mathbf{x}) = \ell(\mathbf{x}) \vee \neg a_{y,j,\ell},$$

so that if the literal is excluded, the expression evaluates to 1, while if it is included, the literal value $\ell(\mathbf{x})$ is enforced. A j -th conjunctive clause for class y is then defined as:

$$c_{y,j}(\mathbf{x}) = \bigwedge_{\ell \in L} \tilde{\ell}_{y,j}(\mathbf{x})$$

Thus, conjunctive clauses relate the inputs – the Boolean literals to their respective TAs and generate 1-bit outputs. This is seen in greater detail through Fig. 2 where the NOT of the TA action is OR’ed with its corresponding literal. This is done for all literals and the resulting AND produces the 1-bit output for each clause. For a classification problem with m classes, there are C clauses per class, evenly divided into positive and negative groups. Positive clauses are marked with a “+” sign. They

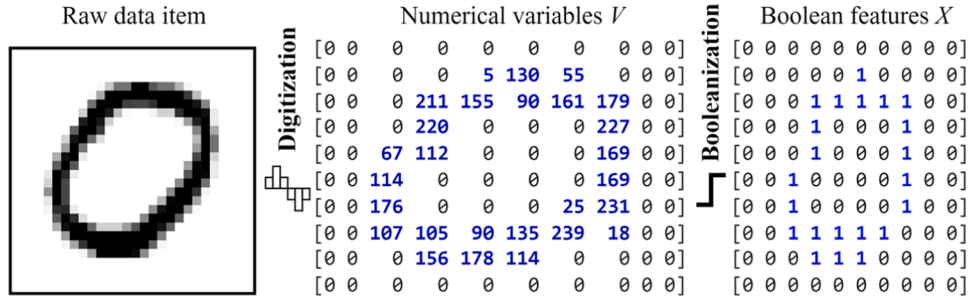


Fig. 3. Example of input features Booleanisation.

are allocated to vote *for* the class. This means they are reinforced to recognise data samples that belong to the class. Negative clauses (marked with “-” sign) are reinforced to recognise data samples that do not belong to the class, i.e., they vote *against* the class.

Output Layer. The output from each clause $c_{y,j} \in \{0, 1\}$ is referred to as a vote, with its contribution weighted by the clause’s polarity. For every class, these votes are aggregated into a class sum:

$$F_y(X) = \sum_{j=1}^{C/2} c_{y,j}^+(X) - \sum_{j=1}^{C/2} c_{y,j}^-(X),$$

and the final prediction is made through the majority voting of class sums as:

$$\hat{y} = \arg \max_y F_y(X).$$

During training, feedback to the TAs is governed by the expected class label, the clause outputs and the current TA actions. The probability of issuing such feedback is modulated by the difference between the set voting threshold T (a TM hyperparameter) and the corresponding clause sum, as detailed in Section 2.5. The memory footprint of a TM model is defined as the total number of TAs used in the TM architecture with m classes and C clauses per class multiplied by the number of Boolean literals $2k$: $C \times m \times 2k$. Each clause contains its own set of TAs (referred to as TA teams). By design, the clause length is bounded by the number of Boolean literals. The only architectural choices are the number of clauses to allocate for each class and the number of states per TA, which together determine the overall capacity and size of the model.

Ultimately, TM offers significant advantages over traditional ML methods in terms of memory, computational simplicity and energy efficiency. Unlike ANNs, which depend on floating-point operations and dense matrix multiplications, TMs rely on efficient bitwise logic (AND, OR, NOT) and simple arithmetic operations such as increment, decrement, summation and majority voting, which eliminates the need for floating-point units and backpropagation of gradient descent, resulting in significantly lower computational complexity. This makes the TM suitable for both training and inference at the edge, where TM-based models can perform classification tasks with up to $20 \times$ lower time and energy consumption than even specialized Binary Neural Networks (BNNs), while maintaining competitive accuracy [13,36].

2.3. Tsetlin machine hyperparameters

As already mentioned, TM requires substantially fewer hyperparameters when compared with conventional ANNs [33]. TM’s hyperparameters can be divided into the following categories (see Table 1):

- Hyperparameters affecting the *architecture* of the TM: the number of clauses per class C and the number of states N within the TA specify the architecture of the TM and thereby the hardware compute and memory resources required.

Table 1

TM Hyperparameters.

Type	Tsetlin Machine Hyperparameters
Architectural	<i>Number of clauses per class, C ($C \geq 2$, even),</i> defines the hardware resources used by the TM in terms of the amount of logic-based clause compute and memory requirements for the TAs in each clause. <i>Number of states in the TA, $2N$ ($N \geq 1$),</i> defines the depth of TA memory.
Algorithmic	<i>Learning sensitivity, s ($s \geq 1$),</i> defines TA’s probability of demotion as $1/s \in (0,1]$ if the associated literal does not contribute to the right clause output. <i>Voting (feedback) threshold, T ($0 < T$),</i> controls the probability of clause’s update as $(T-v)/2T \in [0,1]$, where v is the clauses voting sum for the current data sample.
Training	Number of training epochs

- Hyperparameters controlling TM’s *learning algorithm* and referred to as the learning sensitivity s and the voting threshold T . A pair of T and s largely determines the effectiveness of using TM hardware resources.
- The number of *training epochs* which determines the final accuracy of the TM and the final form of logical clauses, the energy consumed and the time spent to train the TM.

As with DNNs, the TM hyperparameters need to be carefully tuned to obtain the optimal performance. In this regards, T and s are the most important hyperparameters influencing TM accuracy. It was previously shown [20] that optimum T scales with the square root of half the number of clauses as $\sqrt{C/2}$ to ensure the equal voting power [37] across all clauses. Optimum s grows logarithmically with the number of clauses C , resembling the logarithmic relationship between the crowd density and its velocity [38]. Optimum s also largely depends on the data set and needs to be increased to allow the TM to remember fine-grained pattern details when the similarity between classes is large. It is also widely accepted that a higher number of clauses C increase the TM’s information capacity and allow for higher accuracy, given that T and s set optimally.

Sections 4 and 5 contribute to further understanding of optimal hyperparameters and their practical impact on TM’s learning dynamics and its ability to recognise patterns.

2.4. Tsetlin machine learning algorithm

The learning process for the TM involves finding the optimum macro-states for the TAs. This is done through issuing promotion and demotion signals to move the automata state – it is referred to as feedback. Unlike DNN based approach, the TM does not use the predicted output when issuing feedback. Instead, it uses the actual datapoint label, the current TA configuration and clause outputs along with certain stochastic influence to determine how the TA states should be updated. As such, also unlike DNNs, no user defined loss function needs to be

configured. Thus, the TM can be interpreted as a mathematical model of the interactive system where the learning automata make a discrete decision about whether to include or exclude associated features from the clause propositions with respect to the environment – the training dataset. Essentially, the TM learns by memorizing these particular conjunctions of included features that lead to the right classification decision. The TM training process iterates through all input vectors in the dataset multiple times, exploring various feature combinations and improving its memory over the correct actions by receiving reward signals that push each TA deeper into its corresponding states. These decisions are orchestrated by the feedback module.

The feedback module (Fig. 2) dictates the transitions of the TM's system states. More specifically, it rewards TAs of the clauses that correctly classify the input data (i.e., incrementing the TA state deeper into its corresponding action) and penalizes TAs of the clauses that misclassify the result (i.e., decrementing the TA state). This updating process of incrementing or decrementing the state of TAs leads to promoting (deeply remembering) or demoting (deeply forgetting) input literals associated with those TAs. There are two types of TM feedback [9]:

- *Type I* which serves to reward true positive decisions by deeply memorizing literals matching the clause output and forgetting those that do not.
- *Type II* which combats false positives by memorizing false literals previously excluded from the clause.

The feedback requires the use of the two previously mentioned learning hyperparameters T and s . Fig. 4 demonstrates how the feedback decisions descend from the expected class level to the clause level, where the T hyperparameter is used, and then to the level of individual TAs, the updating of which is controlled by the hyperparameter s . Clause level feedback is issued depending on whether it belongs to a “positive” or “negative” polarity group, which are determined by the function $\text{sgn}(C_j)$, and the expected output Y . When the class sum v_i is less than the user-set target T , clauses of the class i are given feedback with the probability $(T - v_i)/2T$. The clauses selected for feedback are then evaluated at the TA level for either Type I or II feedback.

As a part of Type I feedback at the TA level, the TM boosts the promotion of *all true* literals if the clause produces a *True Positive* prediction (*false* literals in this case are demoted with the probability $1/s$) and demotes *all* literals (both *true* and *false*) with the probability $1/s$ if a clause outputs *False Negative*. In this regard, the s hyperparameter whose value may take any floating-point value equal or above 1 is somewhat akin to the learning rate hyperparameter needed for DNNs. Thus, promotion or demotion actions at the TAs level are modulated by probabilities defined by the T and s hyperparameters according to Type I and

Type II feedback as it is shown in Fig. 4.

3. Clause visualization and interpretability

Clauses are logical components of TM that accumulate and represent knowledge by creating conjunctive statements that testify and vote for and/or against each output class. These statements include or exclude certain input literals (i.e., Boolean features X and their negations $\neg X$) depending on the state of the TA associated with each literal. In this and the next sections, we use a variation of the pixel-oriented visualization to interpret the logical rules represented by these clauses and understand how TM clauses create a conjunction of propositional logic depending on the values of T and s hyperparameters.

3.1. Positive clause visual templates

Each positive polarity clause in TM can be viewed as a class template (i.e., a stencil or class pattern) built by aggregating certain class samples pseudo-randomly selected from the dataset. We will show later in this section that, rather than aggregating class samples chosen purely at random, the TM stochastic learning algorithm produces clauses that represent the most common class sub-patterns by finding and clustering similar class samples. Fig. 5 visualises the steady-state positive clauses for class “0” after convergence of a TM with 20 clauses per class trained to recognise handwritten digits, across different values of the s hyperparameter. The model was trained on a subset of MNIST with first 500 samples per class.

When the learning sensitivity s is large, TM clauses become more specific i.e. the clauses start to contain more include TA actions for certain literals such that the clause expressions become more specialized. In the extreme case (e.g., $s \geq 1000$) each clause matches a single class sample, randomly selected from the dataset. The limited number of TM clauses impacts the training accuracy, since true predictions can only be made for a small group of data samples from the training dataset. If the TM has sufficient hardware resources (i.e., when the number of clauses matches or exceeds the size of the training dataset), it will be able to memorize all individual samples, thereby achieving ideal training accuracy. However, being *narrowly specialised* TM clauses will not be capable of making accurate predictions for previously unseen data, which is known as *model overfitting*. When s value decreases, each clause starts to generalize more class samples by “widening” the clause template. This implies fewer included TA actions within the clause, but they correspond to literals found in many samples. Thus, smaller s boosts *generalization ability* of the TM, which increases a proportion of true positive predictions made by each clause on both training and testing datasets. On the other hand, when clause stencils become *too general* (e.g., when $s = 1$), they begin to pass through some data samples which

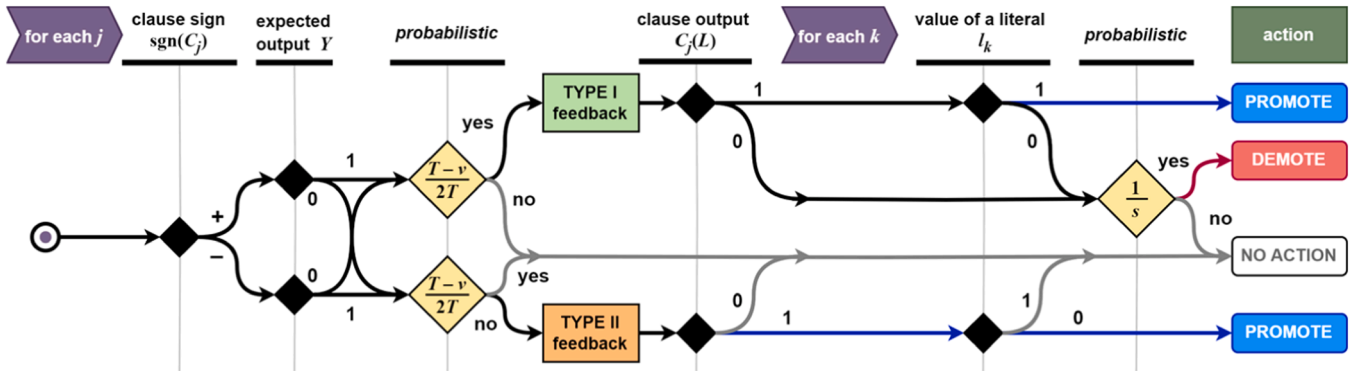


Fig. 4. Single-class TM feedback decision tree. Black diamonds represent choices based on the current TM state, and yellow diamonds represent probabilistic choices, where “yes” is triggered with an indicated probability (i.e., a randomly generated number is less than the specified value) and “no” is triggered otherwise; according to *Boosting of True Positive* variation of Type I feedback all true literals of matching clauses are promoted in order to accelerate learning.

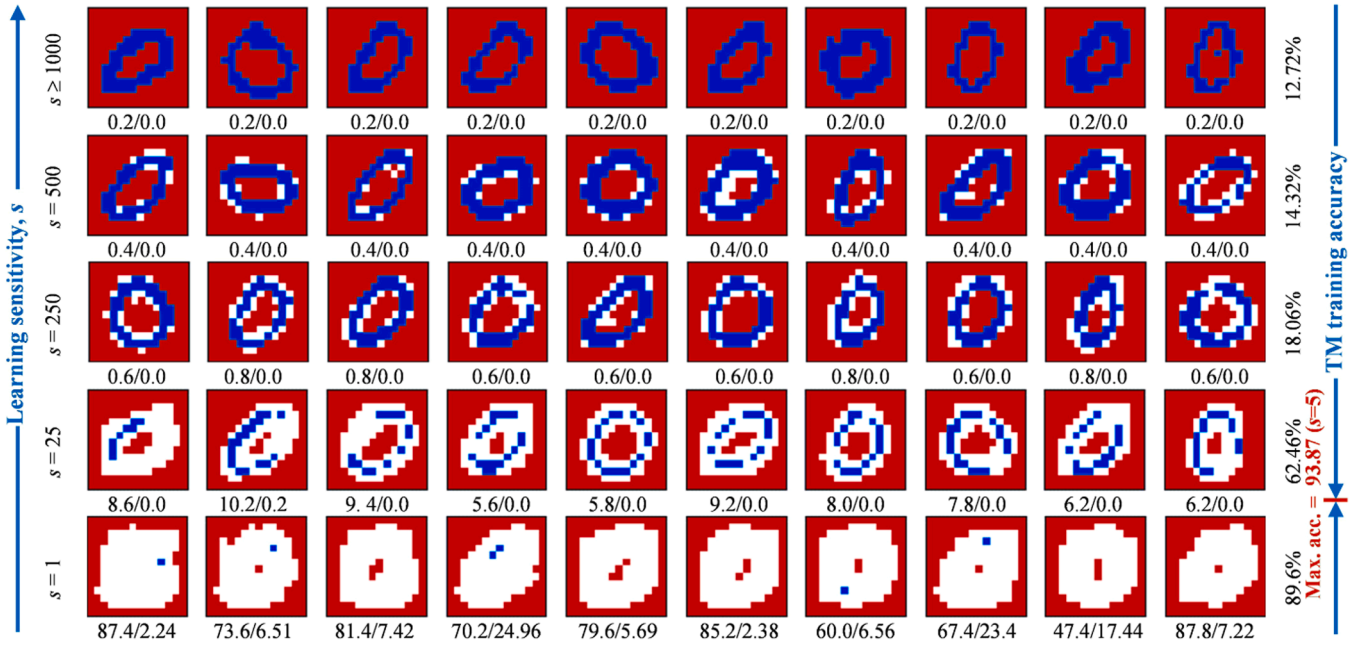


Fig. 5. Visualization of steady-state *positive* clauses of class “0” depending on the s value after 100,000 training epochs. Other TM hyperparameters were set as follows: $C = 20$, $T = 3$ ($T = \sqrt{C/2}$ is the optimal setting according to [20]), $n = 8$ (default setting). Blue pixels correspond to inclusion of positive features X ; red pixels represent inclusion of negated features $\neg X$; white pixels mean exclusion of both positive and negated literals. It is shown that the smaller s , the “wider” TM clauses and the greater their generalization ability. Labels under each clause display a percentage of its true positive and false positive predictions (TP/FP).

belong to other classes and, hence, produce false positive predictions. The effect of learning sensitivity s on the clause’s generalization ability during TM training is shown in Video S.1

3.2. Negative clause visual templates

Negative clauses are used by the TM to recognise data samples which

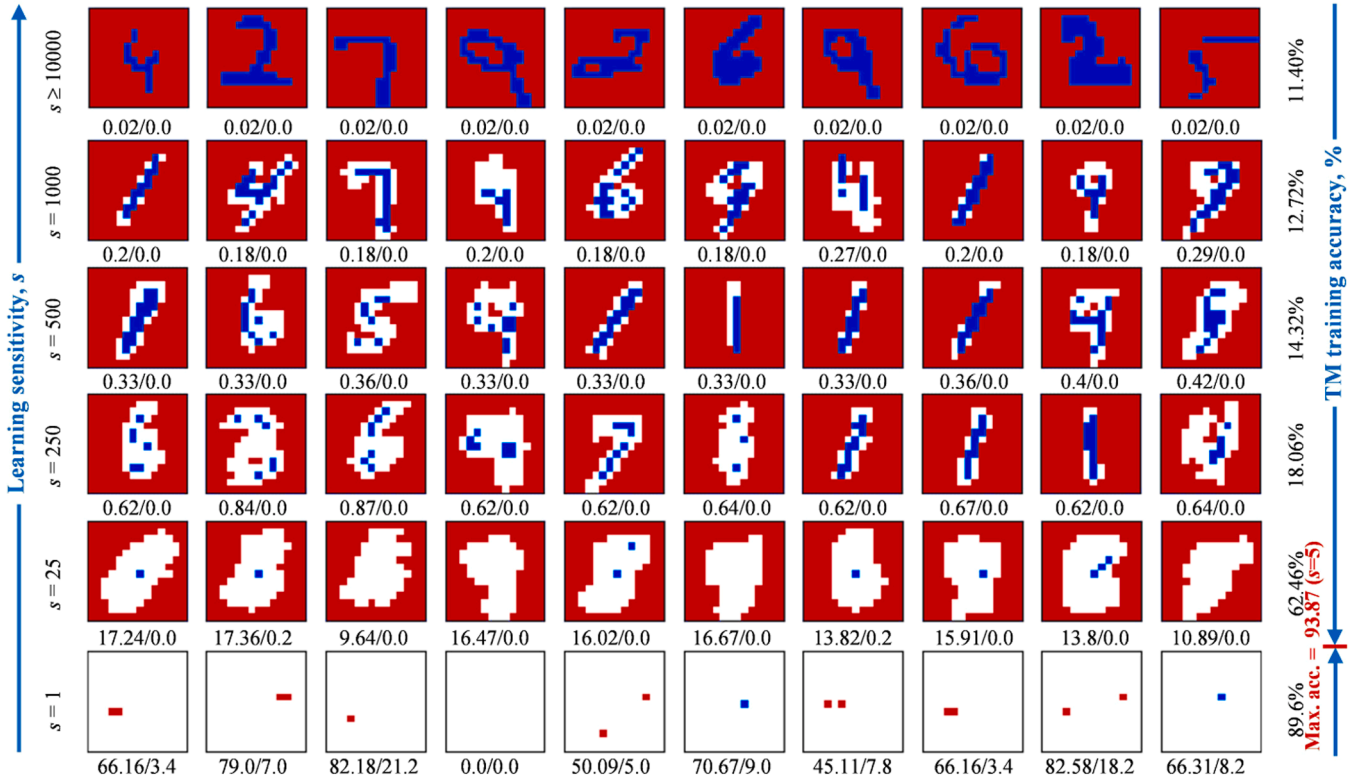


Fig. 6. Visualization of steady-state *negative* clauses of class “0” depending on s value after 100,000 training epochs ($C = 20$, $T = 3$, $n = 8$). It is shown that the negative clauses specialize on counter-classes “1”–“9” (i.e., negative clauses build templates of data samples that *do not* belong to class “0”). Labels under each clause display a percentage of its true negative and false negative predictions (TN/FN). Due to the presence of nine other classes besides class “0”, each negative clause generalizes approximately the same percentage, but proportionately larger number of data samples.

do not belong to the class and to vote against them (i.e., when a clause sum is calculated negative clauses are added with a minus sign). Examples of negative clauses of class “0” are depicted in Fig. 6.

Fig. 6 makes it clear that the negative clauses build templates of the most obvious counter-classes (i.e., classes other than class “0”) and their sub-patterns. The number of counter-samples aggregated by each negative clause also increases with the increase of s . However, because the TM divides clauses allocated per class equally between negative and positive, the information capacity of negative clauses is considerably larger in multi-class classification tasks (proportionally to the number of classes). This can be seen by comparing Fig. 5 and Fig. 6. Negative clause templates are generally “wider” than the templates of positive clauses for the same value of the s hyperparameter, proportional to the total number of classes.

3.3. Clause coverage, true and false predictions

TM clauses can be considered as an array of Boolean filters allocated to each class. When a positive clause sees the data sample that matches the clause template (i.e., all literals of the data sample, which are included in the clause are *true*), it outputs the *True* value, i.e., the clause votes for a data sample to belong to the class represented by the clause. If this is indeed the case, the clause makes a *True Positive (TP)* prediction. When clause templates are considerably “narrow” (which happens when the value of the hyperparameter s is large) and the number of clauses is considerably less than the number of class samples in the training dataset, the clause makes *TP* predictions only for a limited number of class samples, matching its pattern. Thus, the *clause coverage* can be defined as the proportion of *TP* predictions out of the total number of class samples in the training dataset. As the clause template becomes “wider” (when s value is getting smaller), the clause coverage and, hence, the number of *TP* predictions increase. However, some data samples belonging to one of the other classes (i.e., counter-classes) may also begin to match the “wider” clause pattern. As a result, the clause will produce *False Positive (FP)* predictions. Correspondingly, negative clauses that are supposed to output the *True* value when encountering data samples that do not belong to the class can produce *True Negative (TN)* and *False Negative (FN)* predictions. TM makes the final prediction by adding up and comparing all clause votes for and against each class. Thus, the overall TM accuracy depends on the balance between clauses *True* and *False* predictions, i.e., the balance between clauses *specialization* and *generalization*.

3.4. Dataset-Specific optimal learning sensitivity

The learning sensitivity hyperparameter plays a pivotal role in balancing clauses generalization and specialization, directly influencing TM classification accuracy and clause performance, as it is shown in Table 2. The table reports experimental results evaluated on several visual and non-visual datasets after one hundred of training epochs using the TM with one hundred of clauses per class ($C = 100$) and the voting threshold set to its optimal value [20] as $\sqrt{C/2} = 7$. The training and testing accuracies reported in Table 2 were obtained using the standard data splits defined by the dataset creators, with results averaged across 10 runs. The full Python code is available on GitHub: <https://github.com/anatoliy-gorbenko/tm-pattern-recognition>.

For the MNIST and Kuzushiji-MNIST datasets comprising grayscale images of handwritten digits and Hiragana characters respectively, a static thresholding approach was applied using a fixed pixel intensity threshold of 75 across all data samples. The F-MNIST dataset, which contains images of fashion items (e.g., shirts, T-shirts, coats, etc.), was Booleanised using adaptive thresholding based on a weighted Gaussian mean computed over a 101×101 neighbourhood area, giving greater weighting to central pixels; a constant offset $C = 2$ was added to the local threshold to improve foreground-background separation. These

Table 2

Influence of the learning sensitivity s on TM classification accuracy and clause performance tested on different datasets.

TM ($C = 100$, $T = 7, n = 8$); $Epochs=100$	TM accuracy, %		Average clause performance (prediction), %			
			Positive clauses		Negative clauses	
	<i>train</i>	<i>test</i>	<i>TP</i>	<i>FP</i>	<i>TN</i>	<i>FN</i>
MNIST (1568,000 TAs ≈ 1.57 MB)						
$s = 1$	94.40 ± 0.14	92.63 ± 0.18	78.29 ± 0.63	13.67 ± 0.23	42.59 ± 1.87	41.33 ± 1.48
$s = 7.0$ (optimal)	97.00 ± 0.07	96.35 ± 0.09	25.83 ± 1.14	0.98 ± 0.23	27.03 ± 0.96	2.08 ± 0.43
$s = 20$	94.33 ± 0.08	93.49 ± 0.13	21.81 ± 2.03	0.10 ± 0.04	15.69 ± 0.25	0.46 ± 0.32
$s = 100$	52.43 ± 0.37	52.96 ± 0.52	1.87 ± 0.10	0.006 ± 0.001	2.99 ± 0.148	0.014 ± 0.003
Kuzushiji-MNIST (1568,000 TAs ≈ 1.57 MB)						
$s = 1$	86.66 ± 0.20	71.32 ± 0.24	64.8863 ± 0.78	27.86 ± 0.74	46.90 ± 1.33	48.36 ± 0.88
$s = 9.2$ (optimal)	92.65 ± 0.17	80.04 ± 0.42	20.07 ± 0.70	1.96 ± 0.127	19.16 ± 0.42	2.23 ± 0.25
$s = 20$	90.05 ± 0.13	75.68 ± 0.40	17.12 ± 0.77	0.55 ± 0.036	15.41 ± 0.72	0.77 ± 0.18
$s = 100$	53.02 ± 0.66	40.45 ± 0.44	3.46 ± 0.17	0.019 ± 0.003	10.57 ± 0.44	0.049 ± 0.013
Fashion-MNIST (1568,000 TAs ≈ 1.57 MB)						
$s = 1$	85.26 ± 0.13	82.01 ± 0.21	67.15 ± 1.41	19.03 ± 0.63	39.41 ± 1.50	40.44 ± 2.07
$s = 11.2$ (optimal)	86.89 ± 0.15	84.44 ± 0.24	22.91 ± 0.83	0.68 ± 0.06	17.58 ± 0.59	1.07 ± 0.38
$s = 20$	85.91 ± 0.02	83.23 ± 0.23	21.24 ± 1.61	0.31 ± 0.042	14.94 ± 0.71	0.318 ± 0.059
$s = 100$	48.17 ± 1.40	45.85 ± 1.39	5.61 ± 2.58	0.043 ± 0.034	3.93 ± 1.457	0.029 ± 0.019
IMDb (400,000 TAs ≈ 0.39 MB)						
$s = 1$	91.21 ± 0.27	85.36 ± 0.16	47.35 ± 0.40	33.63 ± 0.31	47.21 ± 0.23	33.61 ± 0.14
$s = 1.05$ (optimal)	91.39 ± 0.19	85.37 ± 0.05	46.63 ± 0.38	33.48 ± 0.26	46.61 ± 0.28	33.44 ± 0.21
$s = 20$	87.04 ± 0.24	83.02 ± 0.30	9.15 ± 0.33	2.61 ± 0.24	9.89 ± 0.49	2.98 ± 0.29
$s = 100$	70.65 ± 0.51	68.98 ± 0.74	1.48 ± 0.17	0.39 ± 0.04	1.59 ± 0.39	0.42 ± 0.11
The German Traffic Sign Recognition Benchmark, GTSRB (8806,400 TAs ≈ 8.6 MB)						
$s = 1$	88.99 ± 0.80	81.29 ± 0.70	70.06 ± 1.42	13.79 ± 1.45	33.80 ± 2.56	41.18 ± 2.01
$s = 5.9$ (optimal)	97.87 ± 0.26	89.36 ± 0.24	25.30 ± 1.17	2.09 ± 0.31	27.18 ± 2.21	10.12 ± 2.03
$s = 20$	92.97 ± 0.56	81.02 ± 0.70	13.55 ± 2.28	0.19 ± 0.11	11.62 ± 1.23	1.047 ± 0.44
$s = 100$	38.26 ± 0.54	22.74 ± 0.66	1.24 ± 0.108	0.018 ± 0.005	1.50 ± 0.20	0.28 ± 0.16

parameters were empirically tuned to preserve object shapes while suppressing internal textures and background noise. For the German Traffic Sign Recognition Benchmark (GTSRB) we applied the adaptive mean threshold calculated over smaller 11×11 -pixel sub-regions using the same offset constant. Lastly, the IMDb dataset, consisting of movie reviews, was pre-processed following the method described in [32], where raw text reviews were tokenised into unigrams and bigrams. Frequently occurring phrases were mapped to binary features, resulting in sparse bit-vector representations indicating the presence or absence of selected phrases. Chi-squared feature selection was used to retain the top 1000 most informative features.

Table 2 highlights the need for careful tuning of the s -value based on the intra-class variance and inter-class similarity of the datasets to achieve optimal performance. For the MNIST dataset with relatively well-distinguishable class patterns, the optimal value of $s = 7$, estimated experimentally [39], provides 97 % test accuracy after 100 epochs.

At this setting, TM clauses maintain a high rate of true predictions ($TP=25.83$ % for positive clauses and $TN=27.03$ % for negative clauses, on average) while minimizing false predictions at the level of $FP=0.98$ % and $FN=2.08$ % respectively. The Kuzushiji-MNIST dataset, with greater

intra-class variance, benefits from a slightly higher optimal $s = 9.2$. This enables TM to achieve 80.04 % test accuracy (which is less than for MNIST dataset due to lower clause coverage) maintaining considerably low rate of false predictions: $FP=1.96$ % and $FN=2.23$ %. The best balance between clauses true and false predictions for the Fashion-MNIST dataset, characterised with considerable inter-class similarities, is achieved when the learning sensitivity has even higher value $s = 11.2$.

Too small s (e.g., $s = 1$) leads to excessive generalization, reducing accuracy due to high rate of *False* predictions (i.e., FP and FN rates), observed across all datasets. Learning sensitivity which is too high (e.g., $s = 100$) causes clause over-specialization. On the one hand it results in a significant drop in clause *False* prediction rates. On the other hand, this also leads to considerable reduction of the prediction accuracy due to significant under-coverage of class samples in the dataset. These findings illustrate that the optimal value of the learning sensitivity depends on the dataset's characteristics, with smaller s suitable for datasets with low variance between data samples of the same class and higher s better for datasets with significant inter-class similarity.

Learning sensitivity s may also have important practical applications beyond optimizing TM classification accuracy. In Natural Language Processing tasks datasets often exhibit substantial linguistic diversity within classes and overlapping patterns across classes. For instance, positive and negative sentiments in IMDb reviews may share neutral terms or phrases. By adjusting s , the TM can adapt its clause behaviour to address this complexity. Lower values of the learning sensitivity encourage generalization, allowing the TM to identify broad patterns, such as common sentiment-indicating words. Conversely, higher

learning sensitivity enhances specialization, allowing TM to focus on nuanced differences, such as context-specific sentiment (e.g., “bad” in “bad weather” vs “bad movie”), which was first noticed in [18].

The TM also demonstrated competitive performance on some real-world visual data, such as the GTSRB, while providing interpretable pattern visualisation [40]. Fig. 7 shows that setting the s hyperparameter in the range [20,25] helps eliminate background noise from the learned patterns of traffic signs, improving their relevance. Decreasing s to [5,6] enables the TM to focus on the most consistent features across varying lighting conditions and viewing angles, resulting in optimal performance with each clause generalising across approximately 30 % of class samples. Further reduction of s towards 1 increases clause generalisation to 73 %, but oversimplifies the patterns, omitting critical class-specific features and leading to a rise in false positives up to 7 %, thus impacting the overall classification accuracy.

Ultimately, across all datasets, the TM exhibited a consistent response to variations in the s hyperparameter. At too high s , clauses tend to overspecialise by *memorising* individual class samples. When the number of clauses is limited, this leads to overfitting and poor coverage of the dataset, resulting in a low true positive rate. Conversely, too low value of s causes the TM to overgeneralise class patterns, which increases the rate of false predictions due to insufficient specificity.

4. Clause aggregation and clustering capability

Each TM clause creates a template by aggregating (clustering) the certain number of class samples selected from the training dataset, as

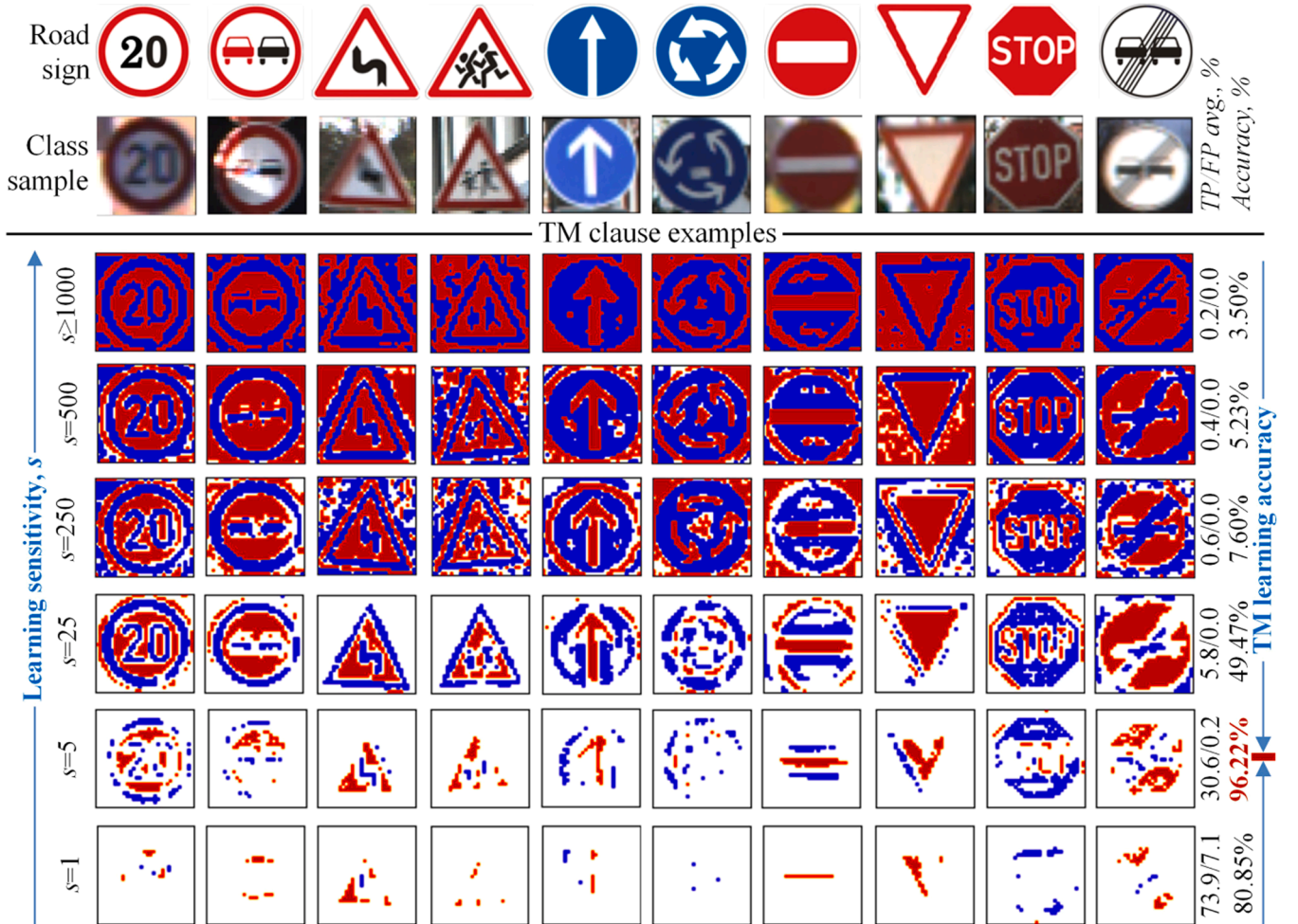


Fig. 7. Example of TM's interpretable pattern visualisation (one random clause is shown for selected classes) of traffic signs depending on the s value. The TM ($C = 20$, $T = 3$, $n = 8$) was trained for 100,000 epochs on a training subset of the GTSRB, limited to 500 samples per class across 43 classes.

shown in Figs. 5–7. The number of class samples ultimately aggregated by each clause (or, more generally, their proportion of the total) depends on the s hyperparameter. For example, for $T = 3$ and $s \geq 750$, each positive clause picks exactly one class sample (i.e., 0.2 %) from the training dataset of 500 handwritten digits per class used in our experiments. When $375 \leq s < 750$, each clause clusters two class samples (i.e., 0.4 % of 500), etc. A model that estimates clause coverage (i.e., a fraction of class samples that each clause aggregates) depending on T and s hyperparameters is introduced and discussed in Section 5. This section examines clause template's building process and TM's inherent clustering capability.

During training, each clause remembers (i.e., promotes toward inclusion) those literals that are common for all clustered class samples and forgets (i.e., demotes toward exclusion) literals that are different.

Once training converges, clauses form the steady-state patterns by AND-ing aggregated samples, as shown in Fig. 8. The clause template is formed by *excluding* non-matching literals and *including* only those literals which are true across all class samples aggregated by that clause. In Fig. 8 we can also see that TM clauses conjunct groups of class instances that are similar to each other. Indeed, TM feedback mechanism is designed in a way that allows clauses to find and cluster class samples which differ in a smaller number of literals as compared to other class samples. This is explained by the following reasoning. At the beginning of training, all TAs are initialized in a weak exclude state. Consequently, clauses templates are blank (i.e., none of positives or negatives literals are included into the cause) and each clause outputs *True*. Once training begins, positive clauses remember the first seen class sample (subject to

$(T - v)/2T$ probability) following the Type I *boost promotion* rule [9].

When TM encounters other data sample of the same class, a clause that previously remembered the first seen class sample produces the false output. This triggers the Type I feedback, which results in the demotion of some TAs (and, hence, removing from the clause some literals if states of respective TAs cross the decision boundary) randomly selected with the probability $1/s$. This, in turn, leads to the erosion of the initial clause template. If s is large, such erosion progresses incrementally, demoting (i.e., excluding from the clause) only a small fraction of literals every time when a new class sample does not match the clause template. Demotions stop as soon as the TM encounters some other class sample (similar to the original one), which matches the “eroded” clause template. The clause template is then reinforced by promoting true literals. This cycle is repeated until the number of demotions is compensated by the number of promotions. The stochastic nature of literal demotions causes each clause to evolve differently. Ultimately, the TM clauses find stable sub-patterns corresponding to groups of similar class samples. This means that the TM is capable of data clustering, which can be further used for unsupervised or semi-supervised learning. This also explains the ability of negative clauses to independently find stable counter-class patterns (see Fig. 6). As s decreases, sample closeness becomes less of an important factor. Finally, if $s = 1$, each clause begins to combine class samples chosen completely at random until $\frac{T-v}{2T}$ is higher than zero according to Type I feedback rule. If T is large, clauses will be motivated to aggregate every other class sample ($\lim_{T \rightarrow \infty} \frac{T-v}{2T} = 0.5$) but are still subject to randomization.

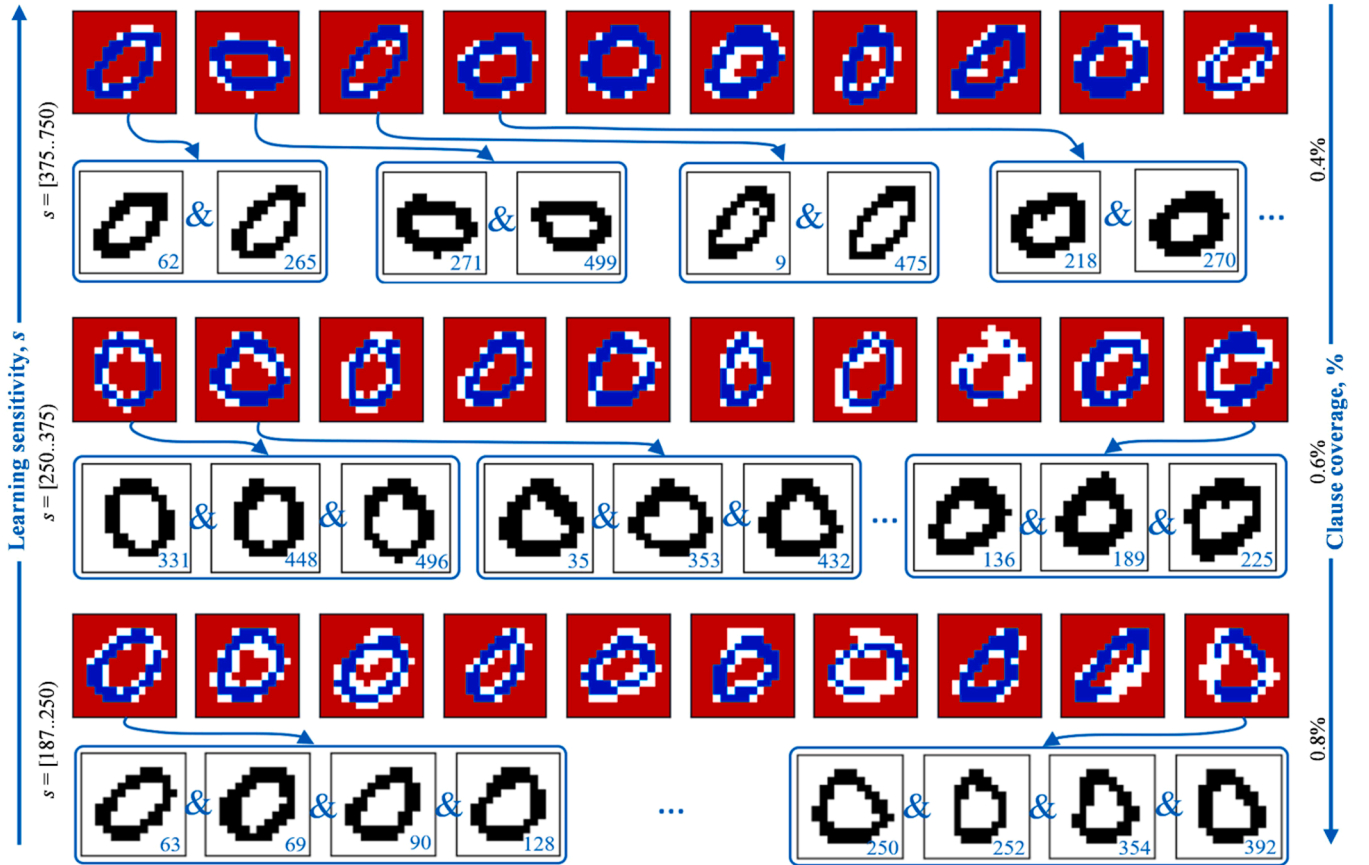


Fig. 8. Clause coverage and template building process depending on s -hyperparameter ($C = 20$, $T = 3$, $n = 8$). During training, each TM clause aggregates a certain number of class samples selected from the dataset. This number (i.e., the clause coverage) depends on the s hyperparameter. For instance, if $s \geq 750$, each positive clause template consists of exactly one class sample randomly selected from the dataset. When s value is in the range $[375, 750]$ each clause clusters exactly two class samples, which is 0.04 % of 500 samples of Class “0” in the used training set, etc. The clause template is ultimately built by including only those positive (blue pixels) and negated (red pixels) features, which are true for all samples conjunct by the clause. Non-matching literals (white pixels) are excluded from the clause. As such, clauses are built by *literal-wise ANDing* of aggregated class samples (sample indexes are shown in the bottom right corners).

5. Impact of hyperparameters

Voting threshold T and learning sensitivity s are the main hyperparameters that control TM learning process and influence the construction of TM clauses. In this section we study how these hyperparameters determine the trade-off between clause specialisation and generalisation and affect classification performance, dataset coverage and the rate of *True* and *False* predictions.

5.1. Trade-off between clause specialization and generalization

When clauses are narrowly specialised, they make true positive predictions with the high confidence, but only for the limited number of class samples precisely matching clause patterns. The TM accuracy remains low as many of data samples remain uncovered by the limited number of specialised clauses. Decreasing learning sensitivity s towards 1 improves clauses generalisation. This means that clause templates become wider and begin to cover more class samples for which they produce true predictions, leaving fewer data samples without votes. Clause templates also begin to overlap, so the overlapping class samples receive more votes from several clauses. At some point, further decrease of s hyperparameter leads to an issue when clauses become “too broad” and begin to cover samples of other classes. From this point on, clauses face a growing push back from Type II feedback (see Fig. 4) in response to the increased number of false positive predictions. Thus, when properly tuned, the s hyperparameter maintains an optimal balance between clause generalisation (i.e., their ability to generalise more class samples) and specialization (i.e., clause’s ability to distinguish between samples of the certain class and other classes). The relationships between TM training accuracy, the average clause coverage of class samples (TP predictions) and counter-class samples (FP predictions), and the overall dataset coverage measured experimentally are shown in Figs. 9–12.

Fig. 9 suggests that the dataset coverage is reversely proportional to the s value. The maximum training accuracy of TM using 20 clauses per class (10 of which are positive ones) reaches 94.36 % for $T = 3$ and $s = 5$ (see Fig. 10). These settings provide the average clause class coverage (Fig. 11) at the level of 0.28 (i.e., 28 %), which ensures 96.42 % of the overall dataset coverage, i.e., only 3.58 % of class samples remain uncovered by any of the positive clause templates. The proportion of clause false positive predictions averages 0.08 %. Further increase of the average clause coverage (by decreasing the value of s toward 1) reduces TM accuracy despite the overall increase of the dataset coverage due to the raising number of FP predictions made by each clause, as it is shown

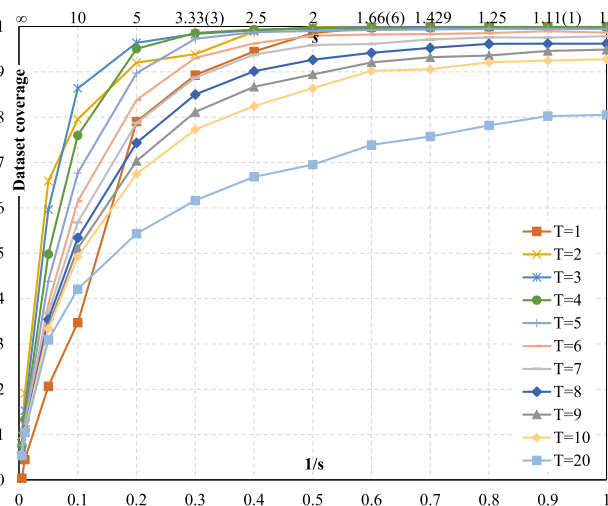


Fig. 9. Dataset coverage depending on T and s ($C = 20$).

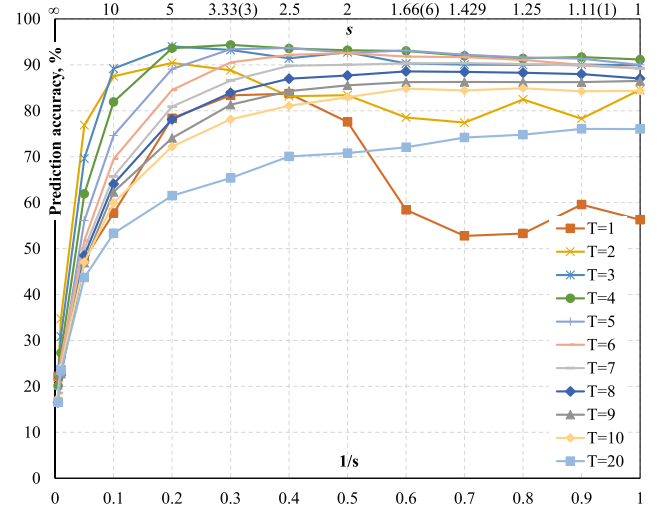


Fig. 10. TM accuracy depending on T and s ($C = 20$).

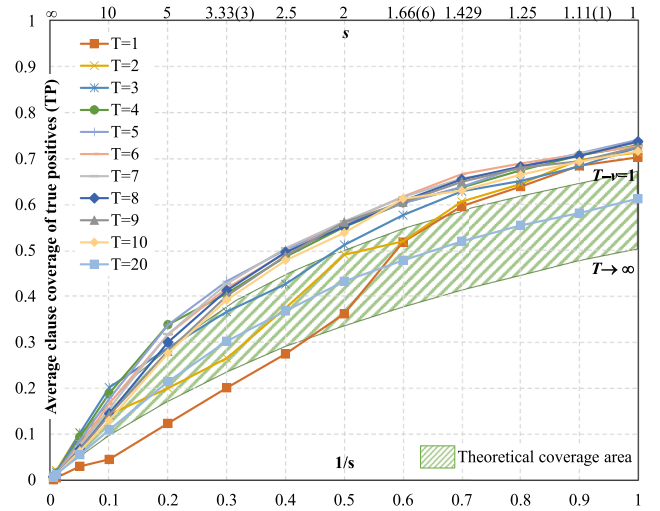


Fig. 11. Average positive clause coverage of class samples (i.e., a proportion of True Positives) depending on T and s ($C = 20$).

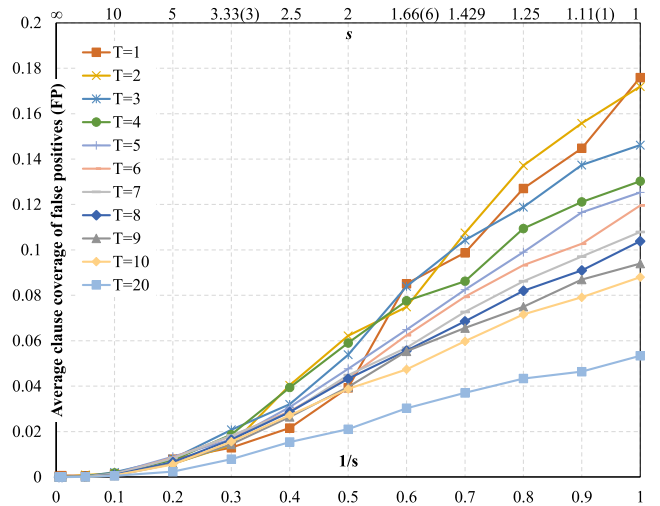


Fig. 12. Average positive clause coverage of counter-class samples (i.e., a proportion of False Positives) depending on T and s ($C = 20$).

in Fig. 12.

On the other hand, increasing the s value makes the clauses more specific and nullifies false positives. However, this also decreases classification accuracy, as overall dataset coverage rapidly declines (Fig. 9) due to decreasing proportion of clauses' TP predictions (Fig. 11). It is also worth noticing that the voting threshold T controls clause diversity, i.e., the degree of clause overlapping. Fig. 9 shows that the larger T is, the more TM clauses overlap each other and, therefore, the lower the overall coverage of the dataset.

5.2. Clause coverage model

Clause coverage (i.e., the number of class samples matching the clause template) can be considered as a quantitative measure of clause generalisation ability. It was shown that hyperparameter s defines a proportion of class samples covered by each clause. Analysis of reinforcement rules and experimental results reported in Figs. 9–12 suggest that if a positive clause only receives Type I feedback, the clause coverage approximates to:

$$CCov = \frac{1}{\alpha s + 1}, \text{ where } \alpha = 1 - \frac{1}{T - v + 1}, \quad (1)$$

given that $T > v$ (otherwise, Type I feedback ceases).

When $T - v = 1$, each clause covers every $\frac{1}{0.5s+1}$ class samples. This value converges to $\frac{1}{s+1}$ if $T \rightarrow \infty$. If $s = 1$, the theoretical clause coverage varies between 0.5 and 0.666 (i.e., each clause covers between 50 % and 66.6 % of class samples on average) depending on the set voting threshold T and the class sum value v . In practice, the actual clause coverage can be somewhat higher (as it is shown in Fig. 11) because clause patterns, when they become significantly “wide”, begin to cover additional class samples matching the clause template even if they were not initially involved in aggregation, including data samples belonging to other classes. Type II feedback acts here as a mechanism to mitigate the effect of increasing number of false positive predictions (see Fig. 12) caused by clause over-generalization if s value is too small.

5.3. Optimal voting threshold, learning sensitivity and the average class sum

During TM training, each class sample receives as many votes from positive clauses as many clause patterns it matches. In the beginning of training the voting sum per each class sample can be very uneven (see Fig. 13,a). The Type I feedback drives training towards an equilibrium where all class samples receive the equal number of votes, approximated to the threshold value T as it is shown on the voting sum histograms (Figs. 13,b and 14,a). The voting threshold T controls TM's confidence in the voting results. When the number of clauses is limited, attaining a higher voting sum necessitates “widening” clause patterns so that each clause covers more samples. On the other hand, it inevitably increases the number of FP predictions, which in turn causes a push back from the Type II feedback (Figs. 13,b and 14,a). If T is too large (Fig. 12), the TM decreases clauses diversity, so that the clauses become largely overlapping, attempting to increase the voting sum v for the covered samples towards T . This also reduces the overall dataset coverage which can be noticed in Fig. 9 and, hence, affects the classification accuracy (i.e., reduces the number of True Positive predictions).

The *balanced* T and s hyperparameters guide the TM training process towards the equal distribution of clause votes across all class samples at the level of T , as it is shown in Fig. 14. In order to ensure that each class sample receives no less than T votes on the average, the product of positive clauses C^+ and clause coverage $CCov$ must be equal or be greater than T : $0.5C \cdot CCov \geq T$. This has two major implications. First, considering that the maximal theoretical clause coverage, achieved when $s = 1$, varies between 0.5 and 0.666 according to (1), it is not practical to set T higher than $0.333C$. Otherwise, the specified threshold will not be reached even theoretically for all class samples. On the other hand, when the threshold value T is too low, only a small number of clause votes are required to reach the voting threshold and, hence, to terminate Type I feedback. This leads to low confidence in the voting results and causes sub-optimal use of hardware resources due to emergence of many “weak” clauses, which are rarely participate in voting (i.e., never or very

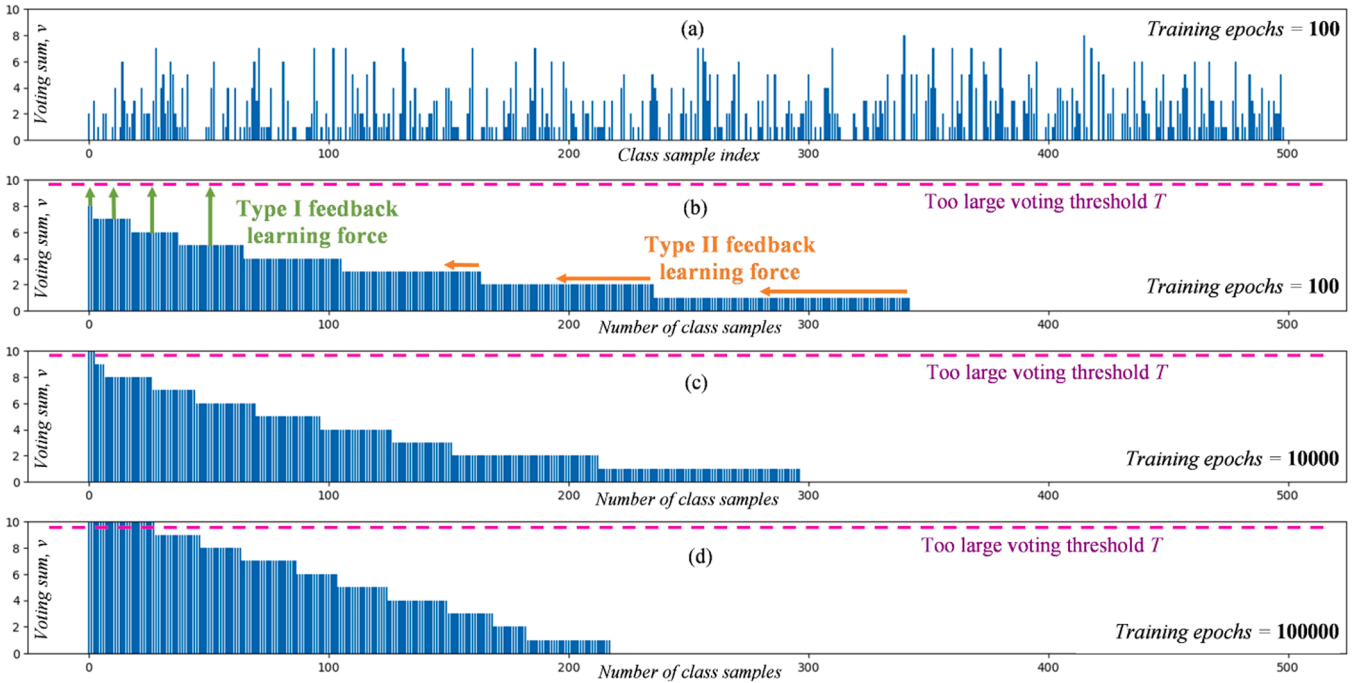


Fig. 13. An example of dataset coverage and learning forces directing TM training towards reaching the equilibrium of voting sums v when the voting threshold T is too large: (a) Class “0” samples coverage and voting sum v per sample after 100 training epochs; (b) voting sum histogram (voting sums are sorted in descending order) and learning forces (Type I and Type II feedbacks) driving the learning process when T is too large after 100 training epochs; (c) voting sum histogram after 10,000 training epochs; (d) voting sum histogram after 100,000 training epochs.

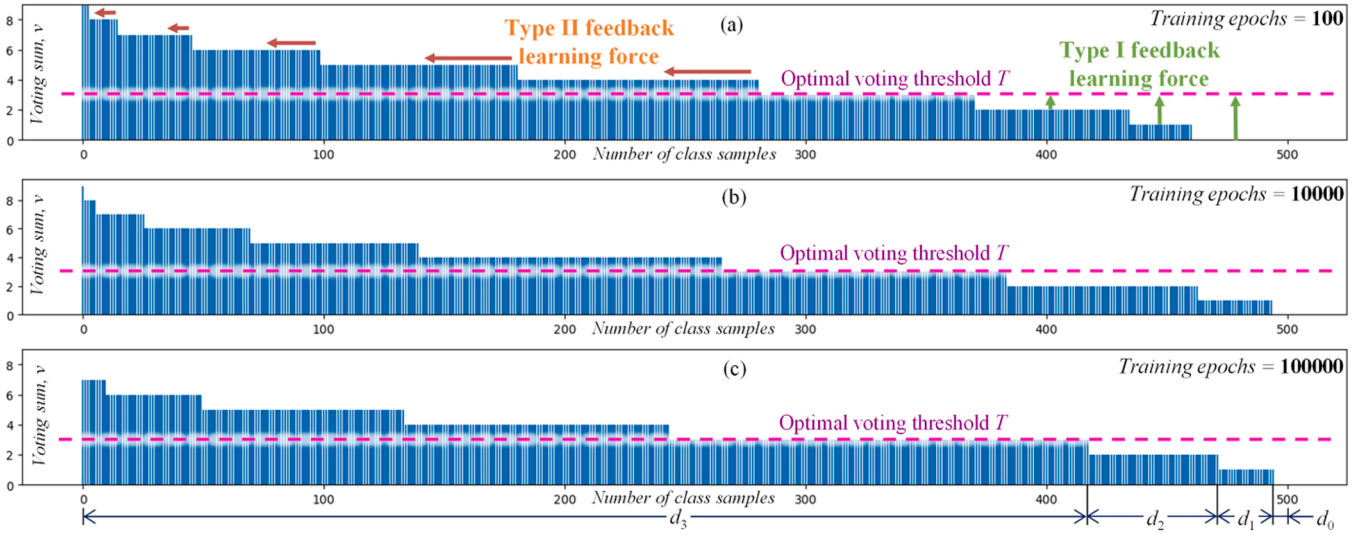


Fig. 14. An example of dataset coverage and learning forces directing TM training towards reaching the equilibrium of voting sums v when the voting threshold T is set optimally ($T = 3$): (a) Class “0” voting sum histogram and learning forces (Type I and Type II feedback) driving the learning process after 100 training epochs; (b) voting sum histogram after 10,000 training epochs; (c) voting sum histogram after 100,000 training epochs; d_3 – is the number of class samples which received $T = 3$ or more clause votes; d_2 , d_1 and d_0 – are numbers of class samples which received $(T-1)=2$, $(T-2)=1$ and $(T-3)=0$ clause votes respectively.

rarely produce true positive votes).

Second, the optimal learning sensitivity s should not be higher than $\frac{C}{\alpha T}$. Depending on the value of T this can be formally specified as following:

$$1 \leq s_{\text{optimal}} \leq \frac{(0.5C - T)(T - v + 1)}{T(T - v)}. \quad (2)$$

This approximates to $(C - 2T)/T$ when $T - v = 1$, i.e., right before Type I and II feedback completely terminates according to TM’s learning algorithm (Fig. 4). The exact value of the optimal s is highly dataset-dependent and must be tuned empirically. In datasets with substantial intra-class variability, balancing clause votes evenly across all samples can be difficult, and lowering s in these cases will help improve the overall dataset coverage. Conversely, if s is set too low, particularly when different classes share many similar features, the TM may incur more false predictions, degrading the overall accuracy. Eqs. (1–2) are in line with experimental results reported in Figs. 9–12 and support previous observations that for any T there exists s (likewise, for any s there exists T) for which the local maximum of accuracy is achieved [41]. They are also aligned with the previous findings showing that the global optimum of the voting threshold T for the given number of clauses C approximates to $\sqrt{C/2}$. This value maximises voting power of each clause voting for the certain class according to Jagiellonian compromise [41] for qualified majority in the Penrose’s voting system [37].

6. TM convergence, Nash equilibrium and training completion

When the sum of the positive and negative clause outputs v reaches the voting threshold T for all data samples, the TM will stop updating TAs under Type I rule. If herewith TM clauses remain sufficiently specific and do not cover data samples from other classes, then Type II feedback also terminate. TM’s ability to converge over an infinite time horizon was proven mathematically [22] using non-linear logic functions like XOR. However, for large data sets TA’s convergence can take unacceptably long time or consume inappropriately large amount of computing resources or energy. Moreover, this might not be achieved even in theory if TM hyperparameters are set sub-optimally (e.g., when the voting threshold T is considerably greater than its optimal value [22]). In contrast to TAs (i.e., TM’s *short-term memory*), where the states are highly volatile, TM clauses can be considered as a *long-term memory*.

Thus, in addition to convergence at the level of TAs it is worth considering the TM convergence at the level of clauses and its ability to reach Nash equilibrium [9].

The TM is built on a game-theoretic foundation where each TA participates in a game to decide whether to include or exclude the specific feature in propositional logic clauses. As a result, each automaton is rewarded in a way that aligns individual interest (i.e., getting positive reinforcement) with the group interest (i.e., building a clause that solve the pattern recognition problem). A Nash equilibrium is reached once the clause achieves a self-consistent configuration, when no single automaton has an incentive to deviate from its current macro state (i.e., *Exclude* or *Include*) despite TAs micro-transitions (i.e., transitions which do not result in crossing the decision boundary) continue to occur. This is also consistent with earlier works that studied learning automata operating in stationary stochastic environments and showed their ability to achieve an optimal state [27,28]. Such equilibrium is observed when TAs that are in the “include state” on average receive more or the same number of promotions (when a clause encounters class samples matching its template) as the number of demotions (when the class sample does not match the clause template) during each training epoch, subject to $(T-v)/2T$ and additional $1/s$ (in case of non-matching samples) probabilities. Conversely, TAs which are in the “exclude state” should receive equal or more demotions than promotions during the training epoch. Therefore, TAs that reach an extreme include or exclude state do not stay there but rather continue to “bounce-off” [21], as it is shown in Fig. 15.

For example, for features that are true for all aggregated class samples and, therefore, are included in a clause (e.g., red and blue pixels in Fig. 5), the equilibrium is reached when:

$$\sum_{i=0}^T \frac{d_i i (T - i)}{2T} \geq \sum_{i=0}^T \frac{d_i (C - i) (T - i)}{2Ts}, \quad (3)$$

where d_i is the number of class samples, which received i clause votes; d_T also includes class samples received the number of clause votes equal or more than T (see Fig. 14,c). The opposite inequality (without accounting Type II feedback triggered by false predictions) must hold for features that are excluded from the clause.

Fig. 15 illustrates that over time TM clauses settle into a stable state, i.e., a Nash equilibrium, where the two counter-acting forces balance out, preventing any TA from crossing the decision boundary despite

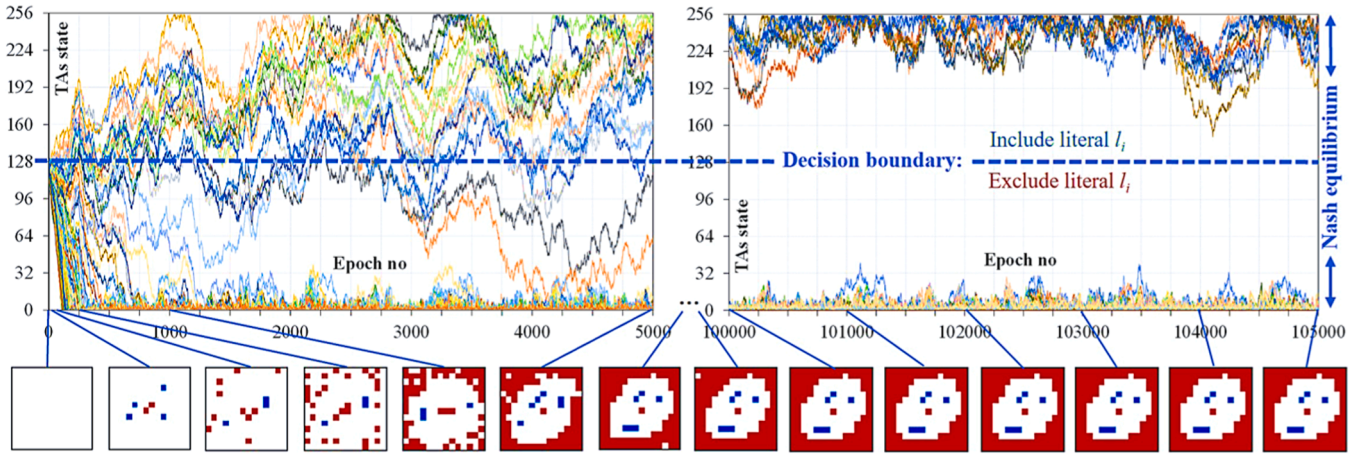


Fig. 15. Evolution and TA states progression of a clause of Class “0” during TM training; Video S.2 demonstrates the initial period of evolution of TM clause patterns in dynamics.

continued state oscillations. Figs. 5 and 6 show examples of these converged clause states under different learning sensitivity settings. These results experimentally confirm ability of a TM to reach a Nash equilibrium, further validating its game-theoretic foundation provided in [9]. Attaining a Nash equilibrium between the promotions and demotions of TAs across all TM clauses formally concludes training as no further clause updates occur. However, this can still require many cycles, especially for datasets with high intra-class variability. In practice, it could be more efficient to stop training once the number of TA transitions crossing the decision boundary per epoch falls below a pre-defined threshold, e.g., 5 %.

Moreover, as in economics or multi-agent systems, a Nash equilibrium does not guarantee TM reaching the absolute best configuration for global classification performance. Therefore, using test accuracy as an early stopping training criterion is practically justified in order to save computational resources and energy.

Regarding TM’s ability to achieve Nash equilibrium, the depth of TAs states creates an oscillation range which can withstand a certain volatility between data samples of the same class. The optimal number of TA states $2N$ thus depends on the size of the dataset and its intra-similarity. Small number of TA states can prevent TM clauses from converging to the stable pattern especially if the dataset is large or when there is a considerable heterogeneity between class samples. Too large N results in an increase in the number of epochs required for TM to converge and, ultimately, in longer training time and higher energy consumption [20].

7. Summary of key findings

Our study highlights several key observations that contribute to a deeper understanding of TM’s performance and learning dynamics. Firstly, we demonstrated that the TM’s accuracy and pattern recognition capabilities are strongly influenced by two critical hyperparameters: the voting threshold T and the learning sensitivity s .

The voting threshold T should ideally scale with the square root of the number of positive clauses, following Penrose’s law, to ensure equal voting power across clauses in the TM’s majority-vote architecture. The learning sensitivity hyperparameter s , on the other hand, requires careful tuning based on dataset characteristics and the number of clauses allocated per class. For configurations with a small number of clauses, the clause templates must be more general to effectively cover the dataset. This generalisation is achieved by reducing the value of the s hyperparameter. However, this may lead to increased *False* predictions, especially noticeable for datasets with high inter-similarity between classes, like F-MNIST. Conversely, when the TM employs a large number of clauses C , higher accuracy can be achieved by increasing the s -value,

which makes the clauses more specific. This will reduce the probability of *False* predictions while maintaining the dataset coverage. It was also shown that the optimal learning sensitivity s is upper-bounded by a certain estimable value that, on average, ensures the specified voting threshold T is achieved across all data samples. Thus, the s hyperparameter serves as a crucial control for balancing clause generalisation and specificity, and its optimum can be determined considering the desired ratio between Type I and Type II errors, the number of used clauses C and the set voting threshold T . Since T and s are global TM hyperparameters, balanced clause development during training depends on equal TAs reinforcement across classes. This requires the use of class-balanced datasets, the application of oversampling strategies or, alternatively, introducing class-specific (T , s)-tuning to ensure equitable learning dynamics.

Ultimately, TM clauses are constructed by ANDing the certain number (depending on the s hyperparameter) of similar class samples from the training dataset and function as class-specific filters, akin to stencils, that fires (outputs a true value/a vote) when an input sample exactly matches the clause stencil. The class with the maximum sum of clause votes is then returned as TM prediction. In this regard, the TM can be considered as an array of Boolean filters bringing a close analogy between the TM and a concept of active phased array antenna in digital signal processing.

Although automata-level convergence when no further TA updates happen has been already theoretically established, we speculate that convergence at the clause level, i.e., ability to reach a Nash equilibrium among clauses such that no further decision-boundary crossings occur despite TA states continuing to oscillate near one of the two extremes, may offer a more practical and resource-efficient criterion for terminating training. By eliminating unnecessary iterations, it can reduce training time and energy consumption, which is critical for deployment on resource-constrained edge hardware.

Finally, our results demonstrate the ability of TM clause teams to self-organise in stationary stochastic environments and capture persistent sub-patterns within each class by grouping similar samples (i.e., samples that differ in only a small number of literals), where a higher s -value enforces closer similarity. This behaviour highlights the TM potential for future applications in unsupervised and semi-supervised learning scenarios.

8. Conclusion

This paper examines learning dynamics, interpretability and pattern recognition performance of the TM, a relatively new ML algorithm based on the collective behaviour of learning automata. To the best of our

knowledge, this work represents the first systematic exploration of the TM's built-in explainability and visualization capabilities and influence of its hyperparameters on clauses *specialization* and *generalization*. While contemporary XAI methods such as LIME, SHAP or attention-based models excel at illuminating why a complex black box model made a particular prediction by fitting local surrogate models or highlighting salient input features for individual samples, their explanations remain inherently sample-specific and often require secondary interpretation to distil global patterns. In contrast, the TM's internal representation is expressed directly as a collection of propositional clauses ("if-then" decision rules) that capture the underlying class patterns and are applied uniformly to all inputs, yielding a naturally global, human-readable model of its decision logic without post-hoc approximation. This rule-based transparency allows to trace how each clause contributes to classification outcomes across the dataset and to visualize clause dynamics as learning unfolds.

The results presented in the paper contribute to a deeper understanding of the TM's hyperparameters and learning dynamics, helping to achieve a robust balance between generalisation and specialisation of clause templates, maximise TM performance and overall classification accuracy while minimising training time and consumption of computational and energy resources. They were validated across a diverse set of visual and non-visual datasets, including MNIST, F-MNIST, K-MNIST, GTSRB, and IMDB, demonstrating the broad applicability and strong potential for wide adoption.

Future research could investigate the TM's capacity to discover and leverage unlabelled sub-patterns by automatically grouping similar samples within a class to unlock new unsupervised and semi-supervised learning opportunities. Besides, refining the core learning dynamics through alternative feedback schemes, adaptive hyperparameters control and smarter clause-initialization strategies could accelerate convergence and boost accuracy without sacrificing TM's efficiency. Finally, integrating semantic preprocessing pipelines or embedding the TM into hybrid neuro-symbolic architectures offers a promising path for scaling its rule-based strengths to the challenges of complex visual datasets, such as CIFAR-100 or ImageNet.

All underlying tools and code are available at <https://github.com/anatoliy-gorbenko/tm-pattern-recognition>.

CRedit authorship contribution statement

Olga Tarasyuk: Writing – original draft, Visualization, Software, Methodology, Data curation, Conceptualization. **Anatoliy Gorbenko:** Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Investigation, Data curation, Conceptualization. **Tousif Rahman:** Writing – review & editing, Validation, Formal analysis. **Lei Jiao:** Writing – review & editing, Validation, Supervision, Methodology, Conceptualization. **Ole-Christoffer Granmo:** Writing – review & editing, Validation, Supervision, Methodology, Conceptualization. **Rishad Shafik:** Writing – review & editing, Validation, Supervision, Methodology, Conceptualization. **Alex Yakovlev:** Writing – review & editing, Validation, Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This research was supported by the British Academy's Researchers at Risk Fellowships Programme (RaR/100289), EPSRC Standard Mode Grant "KNOT" (EP/Z533841/1) and EPSRC Programme Grant "SON-NETS" (EP/X036006/1).

Supplementary materials

Supplementary material associated with this article can be found, in the online version, at [doi:10.1016/j.patcog.2025.113028](https://doi.org/10.1016/j.patcog.2025.113028).

Video S.1. The effect of learning sensitivity s on the clause's generalization ability.

Video S.2. TM learning hand-written digits: evolution of clause patterns.

Data availability

Data will be made available on request.

References

- [1] A. Rahman, T. Debnath, D. Kundu, Md.S.I. Khan, A.A. Aishi, S. Sazzad, M. Sayduzzaman, S.S. Band, Machine Learning and Deep Learning-Based Approach in Smart Healthcare: Recent Advances, Applications, Challenges and Opportunities, 11, AIMS Public Health, 2024, pp. 58–109, <https://doi.org/10.3934/publichealth.2024004>.
- [2] E. Mienye, N. Jere, G. Obaido, I.D. Mienye, K. Aruleba, Deep learning in finance: a survey of applications and techniques, AI 5 (2024) 2066–2091, <https://doi.org/10.3390/ai5040101>.
- [3] A. Hozouri, A. Mirzaei, M. Effatparvar, A comprehensive survey on intrusion detection systems with advances in machine learning, deep learning and emerging cybersecurity challenges, Discov. Artif. Intell. 5 (2025) 1–38, <https://doi.org/10.1007/s44163-025-00578-1>.
- [4] J.J. Zhou, Research on the Complexity Characteristics of Convolutional Neural Networks, in: 2023 IEEE 7th Information Technology and Mechatronics Engineering Conference (ITOEC), IEEE, 2023, pp. 402–405, <https://doi.org/10.1109/itoec57671.2023.10291768>.
- [5] L. Franceschi, M. Donini, V. Perrone, A. Klein, C. Archambeau, M. Seeger, M. Pontil, P. Frasconi, Hyperparameter optimization in machine learning, Found. Trends Mach. Learn. 18 (2025) 1054–1201, <https://doi.org/10.1561/22000000088>.
- [6] C. Rudin, Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead, Nat. Mach. Intell. 1 (2019) 206–215, <https://doi.org/10.1038/s42256-019-0048-x>.
- [7] Y. Pang, H. Wu, C. Wu, Cross-modal co-feedback cellular automata for RGB-T saliency detection, Pattern. Recognit. 135 (2023) 1–15, <https://doi.org/10.1016/j.patcog.2022.109138>.
- [8] X. Hu, W. Pedrycz, X. Wang, Fuzzy classifiers with information granules in feature space and logic-based computing, Pattern. Recognit. 80 (2018) 156–167, <https://doi.org/10.1016/j.patcog.2018.03.011>.
- [9] O.-C. Granmo, The Tsetlin Machine – A Game Theoretic Bandit Driven Approach to Optimal Pattern Recognition with Propositional Logic, ArXiv. (2018) 1–14, 1804.01508.
- [10] M.L. Tsetlin, Automata Theory and Modeling of Biological Systems, Academic Press, New York, 1973.
- [11] V. Varshavsky, D. Pospelov, Puppets without Strings: Reflections on the Evolution and Control of Some Man-Made Systems, Mir Publishers, Moscow, 1988.
- [12] K. Narendra, M. Thathachar, Learning Automata: An Introduction, Dover Publications, 2012.
- [13] S. Maheshwari, T. Rahman, R. Shafik, A. Yakovlev, A. Rafiev, L. Jiao, O.-C. Granmo, REDRESS: generating Compressed Models for Edge Inference Using Tsetlin Machines, IEEE Trans. Pattern. Anal. Mach. Intell. 45 (2023) 11152–11168, <https://doi.org/10.1109/tpami.2023.3268415>.
- [14] G. Mao, T. Rahman, S. Maheshwari, B. Pattison, Z. Shao, R. Shafik, A. Yakovlev, Dynamic Tsetlin Machine Accelerators for On-Chip Training Using FPGAs, IEEE Trans. Circuits Syst. I: Regul. Pap. 72 (2025) 6962–6975, <https://doi.org/10.1109/TCSI.2025.3564875>.
- [15] S.A. Tunheim, Y. Zheng, L. Jiao, R. Shafik, A. Yakovlev, O.-C. Granmo, An All-Digital 8.6-nJ/Frame 65-nm Tsetlin Machine Image Classification Accelerator, IEEE Trans. Circuits Syst. I: Regul. Pap. (2025) 1–14, <https://doi.org/10.1109/TCSI.2025.3586698>.
- [16] S. Jeeru, R. Olsson Omslandseter, P.-A. Andersen, A. James Morrison, N. Sokolova, O.-C. Granmo, L. Jiao, MixCTME: a Mixture of Convolutional Tsetlin Machine Experts Using Diverse Spectrogram Visualizations for Jamming Signal Classification, IEEE Internet. Things. J. 12 (2025) 47667–47678, <https://doi.org/10.1109/JIOT.2025.3606366>.
- [17] J. Lei, T. Rahman, R. Shafik, A. Wheeldon, A. Yakovlev, O.-C. Granmo, F. Kawsar, A. Mathur, Low-power audio keyword spotting using Tsetlin machines, J. Low Power Electron. Appl. 11 (2021) 18, <https://doi.org/10.3390/jlpea11020018>.
- [18] R.K. Yadav, L. Jiao, O.-C. Granmo, M. Goodwin, Robust interpretable text classification against spurious correlations using AND-rules with negation, in: Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, International Joint Conferences on Artificial Intelligence Organization, California, 2022, pp. 4439–4446, <https://doi.org/10.1093/bioadv/vbaf140>.
- [19] O. Tarasyuk, A. Gorbenko, M. Eberl, N. Topley, J. Zhang, R. Shafik, A. Yakovlev, Prediction of the infecting organism in peritoneal dialysis patients with acute peritonitis using interpretable Tsetlin Machines, Bioinform. Adv. (2025) 1–40, <https://doi.org/10.1093/bioadv/vbaf140>.

- [20] O. Tarasyuk, T. Rahman, R. Shafik, A. Yakovlev, A. Gorbenko, O.-C. Granmo, L. Jiao, in: Systematic Search for Optimal Hyper-parameters of the Tsetlin Machine on MNIST Dataset, in: 2023 Int.Symposium on the Tsetlin Machine (ISTM), IEEE, 2023, <https://doi.org/10.1109/istm58889.2023.10454969>.
- [21] A. Rafiev, J. Morris, F. Xia, R. Shafik, A. Yakovlev, O.-C. Granmo, A. Brown, Visualization of Machine Learning Dynamics in Tsetlin Machines. 2022 International Symposium on the Tsetlin Machine (ISTM), IEEE, 2022, pp. 81–88, <https://doi.org/10.1109/istm54910.2022.00020>.
- [22] L. Jiao, X. Zhang, O.-C. Granmo, K.D. Abeyrathna, On the Convergence of Tsetlin Machines for the XOR Operator, IEEE Trans. Pattern. Anal. Mach. Intell. 45 (2023) 6072–6085, <https://doi.org/10.1109/tpami.2022.3203150>.
- [23] R. Saleem, B. Yuan, F. Kurugollu, A. Anjum, L. Liu, Explaining deep neural networks: a survey on the global interpretation methods, Neurocomputing. 513 (2022) 165–180, <https://doi.org/10.1016/j.neucom.2022.09.129>.
- [24] E. Mohamed, K. Sirlantzis, G. Howells, A review of visualisation-as-explanation techniques for convolutional neural networks and their evaluation, Displays 73 (2022) 102239, <https://doi.org/10.1016/j.displa.2022.102239>.
- [25] A.M. Salih, Z. Raisi-Estabragh, I.B. Galazzo, P. Radeva, S.E. Petersen, K. Lekadir, G. Menegaz, A Perspective on Explainable Artificial Intelligence Methods: SHAP and LIME, Adv. Intell. Syst. (2024), <https://doi.org/10.1002/aisy.202400304>.
- [26] O.-C. Granmo, S. Glimsdal, L. Jiao, M. Goodwin, C.W. Omlin, G.T. Berge, The convolutional Tsetlin machine, ArXiv. (2019) 1–14, 1905.09688.
- [27] M.L. Tsetlin, On the behavior of finite automata in random media, Autom. Remote Control 22 (1961) 1345–1354.
- [28] A. Yazidi, B.J. Oommen, G. Horn, O.-C. Granmo, Stochastic discretized learning-based weak estimation: a novel estimation method for non-stationary environments, Pattern. Recognit. 60 (2016) 430–443, <https://doi.org/10.1016/j.patcog.2016.05.001>.
- [29] S. Glimsdal, O.-C. Granmo, Coalesced multi-output Tsetlin machines with clause sharing, ArXiv. (2021) 1–23, 2108.07594.
- [30] K. Darshana Abeyrathna, O.-C. Granmo, X. Zhang, L. Jiao, M. Goodwin, The regression Tsetlin machine: a novel approach to interpretable nonlinear regression, Philos. Trans. R. Soc. A: Math., Phys. Eng. Sci. 378 (2020) 1–14, <https://doi.org/10.1098/rsta.2019.0165>.
- [31] Y. Grønningstær, H.S. Smørvik, O.-C. Granmo, An optimized toolbox for advanced image processing with Tsetlin machine composites, ArXiv. (2024) 1–8, 2406.00704.
- [32] Tsetlin Machine, CAIR, 2018. <https://github.com/cair/tsetlinmachine>.
- [33] O. Tarasyuk, A. Gorbenko, T. Rahman, R. Shafik, A. Yakovlev, Logic-based machine learning with reproducible decision model using the Tsetlin machine, in: 2023 IEEE 12th International Conference on Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications (IDAACS), IEEE, 2023, pp. 465–470, <https://doi.org/10.1109/IDAACS58523.2023.10348711>.
- [34] M. KIMURA, T. OUCHI, H. NAKAHAMA, K. SHIMA, S. SAITO, Learning automaton as a model to simulate and analyze learning behavior in rats, Int. J. Syst. Sci. 19 (1988) 2079–2090, <https://doi.org/10.1080/00207728808964100>.
- [35] T. Rahman, A. Wheeldon, R. Shafik, A. Yakovlev, J. Lei, O.-C. Granmo, S. Das, Data Booleanization for Energy Efficient On-Chip Learning using Logic Driven AI, in: 2022 International Symposium on the Tsetlin Machine (ISTM), IEEE, 2022, pp. 29–36, <https://doi.org/10.1109/istm54910.2022.00014>.
- [36] A. Wheeldon, R. Shafik, T. Rahman, J. Lei, A. Yakovlev, O.-C. Granmo, Learning automata based energy-efficient AI hardware design for IoT applications, Philos. Trans. R. Soc. A: Math., Phys. Eng. Sci. 378 (2020) 20190593, <https://doi.org/10.1098/rsta.2019.0593>.
- [37] L.S. Penrose, The elementary statistics of majority voting, J. R. Stat. Soc. 109 (1946) 53, <https://doi.org/10.2307/2981392>.
- [38] Z. Fang, S.M. Lo, J.A. Lu, On the relationship between crowd density and movement velocity, Fire Saf. J. 38 (2003) 271–283, [https://doi.org/10.1016/s0379-7112\(02\)00058-9](https://doi.org/10.1016/s0379-7112(02)00058-9).
- [39] L. Yang, A. Shami, On hyperparameter optimization of machine learning algorithms: theory and practice, Neurocomputing. 415 (2020) 295–316, <https://doi.org/10.1016/j.neucom.2020.07.061>.
- [40] A. Gorbenko, O. Gordieiev, O. Tarasyuk, R. Shafik, A. Yakovlev, A. Akulynichev, Optimising tsetlin machine for traffic sign recognition: A study of image preprocessing, booleanisation and ensemble methods. 2025 International Symposium on the Tsetlin Machine (ISTM), IEEE, 2025, pp. 86–93. <https://doi.org/10.1109/ISTM67926.2025.00021>.
- [41] K. Życzkowski, W. Słomczyński, Square Root Voting System, Optimal Threshold and π , in: Power, Voting, and Voting Power: 30 Years After, Springer Berlin Heidelberg, 2013, pp. 573–592, https://doi.org/10.1007/978-3-642-35929-3_30.